

Main Concepts of React.js for Frontend Developers

1. JSX (JavaScript XML)

- JSX is a syntax extension for JavaScript, allowing you to write HTML-like code within JavaScript.

React components are written using JSX.

- JSX makes it easier to visualize the UI structure and it gets transformed into `React.createElement()` calls behind the scenes.

2. Components

- Functional Components: Functional components are simple JavaScript functions that accept props (inputs) and return JSX.
- Class Components: Class components are ES6 classes that extend `React.Component` and require a `render()` method to return JSX.

3. Props (Properties)

- Props are read-only properties passed from a parent component to a child component. They allow you to pass data between components.

4. State

- State is mutable data that is owned by a component and can change over time, triggering re-renders.
- Functional components use the `useState` hook to declare and manage state.

5. `useState` Hook

- `useState` is a hook that lets you add state to functional components.

- You can modify the state using the setter function returned by `useState`.

6. `useEffect` Hook

- `useEffect` is a hook that allows you to perform side effects (e.g., data fetching, DOM manipulation) in functional components.
- It is commonly used for things like fetching data or subscribing to external services.

7. Event Handling

- React provides a synthetic event system for handling user events (e.g., clicks, form submissions).
- Event handlers can be passed as props to child components.

8. Conditional Rendering

- React allows conditional rendering using JavaScript operators like ``if``, ``ternary``, or ``&&`` inside JSX to render elements based on conditions.

9. Lists and Keys

- React allows rendering lists of elements using the ``.map()`` method.
- Keys help React identify which items have changed, added, or removed in the list.

10. Forms and Controlled Components

- Forms in React are typically controlled components where form data is handled by React state.

11. React Router

- React Router is a library that allows you to handle navigation between pages in a single-page application.

12. Context API

- The Context API allows you to share data across components without having to pass props manually at each level.

13. React Hooks (useRef, useMemo, useCallback)

- useRef is used to persist values across renders without causing a re-render.
- useMemo and useCallback optimize performance by memoizing values and functions.

14. Error Boundaries

- Error Boundaries are used to catch JavaScript errors in the component tree and display fallback UI without crashing the app.

15. Code Splitting

- Code splitting allows you to split your code into smaller bundles that can be loaded on demand, improving performance.

16. Testing in React

- Testing libraries like Jest and React Testing Library are used to test React components and ensure the reliability of your app.

17. Performance Optimization

- Techniques like memoization, lazy loading, and virtualization can help optimize the performance of React apps.

18. State Management (Redux / Context)

- Redux is a library used for managing global state in React applications, while Context API is used for simpler state management without external libraries.

19. Build and Deploy

- Learn how to build your React app for production and deploy it to platforms like Netlify, Vercel, or GitHub Pages.